

COMP4601

Hardware Acceleration of Canny Edge Detection

Accelerating the full Canny pipeline onto the Kria KV260's programmable logic with Vitis HLS better than real-time.

TEAM · EDGE DETECTIVES

Kaashvi Goel (z5623123) · Rahul Vignesh Kommineni (z5611329) · Luke McCallum (z5591695) · Thomas Saw (z5587733)

Project overview: aims & results

Gray

Blur

Sobel

NMS

Thresh

Hyst

Take Canny off the CPU and prove a hand-built HLS pipeline runs it correctly, live, and better than real-time on the KV260.

WHAT WE SET OUT TO DO

From an OpenCV baseline to a real-time HLS pipeline

- ✓ Baseline: time cv2.Canny() on the PS ARM cores
- ✓ Build all Canny stages as Vitis HLS kernels
- ✓ Verify bit-exact against the OpenCV reference (C-sim)
- ✓ Deploy on the KV260 PL live webcam in, edges out
- ✓ Benchmark & optimise against the software baseline

TARGET ≥ 30 FPS

≈ **272 fps**

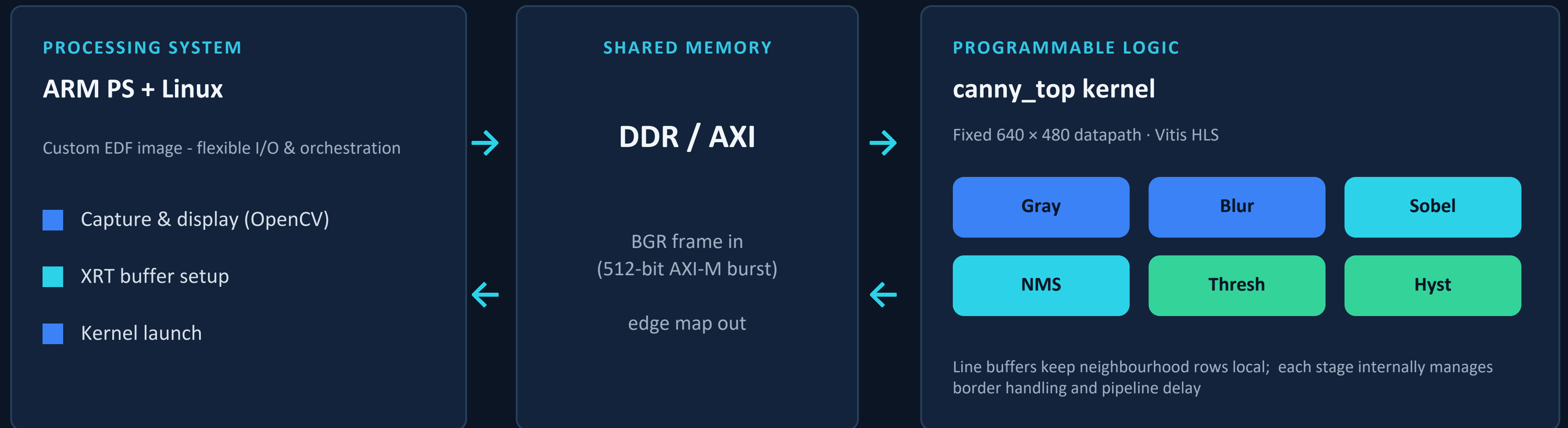
on the Kria KV260 better than real-time

ALSO DELIVERED

- ✓ ≈ 1.5× faster than OpenCV on the PS ARM
- ✓ Bit-exact match to the OpenCV reference
- ✓ Tiny fabric footprint < 28% BRAM used

System design: OpenCV control, HLS acceleration

A clean split: software handles capture/display and XRT control; the PL runs a deterministic Canny pipeline.



Design decision

Keep variable system concerns in software; put repetitive neighbourhood computation into hardware.

Algorithm mapped into row-timed HLS stages

Correctness depended on more than the Canny equations: we managed borders, line buffers and pipeline timing.

OpenCV matching BGR order · reflect Gaussian border · L1 $|G_x| + |G_y|$

Synthesizable static line buffers · no OpenCV in kernel · fixed dims



VERIFICATION BUILT INTO DESIGN

Each stage has a whole-image C++ reference. A full-pipeline test then compares all 307,200 output pixels against our OpenCV-matching reference pipeline, detecting border, buffering and pipeline-delay errors that may not be visible from the output image alone.

TEAM USE OF AI

AI assisted with debugging strategy, testbench structure, the Git/Vitis workflow and explaining HLS trade-offs not with writing the kernels. Every stage was accepted only after native tests, C-simulation and synthesis passed.

Architecture evolution: streaming kernel ready for benchmarking

The final design is not just the Canny stages it is how those stages exchange data and how we proved the stream pipeline before Luke reports results.

DATA MOVEMENT REDESIGN

From row hand-off to FIFO streams

BEFORE

Row-array hand-off

Full rows moved stage-to-stage; overlap was limited.

AFTER

hls::stream dataflow

FIFO communication lets stages overlap after pipe fill.

AXI read

Gray

Blur

Sobel

NMS

Thresh

Hyst

AXI write

✓

DATAFLOW + streams: frame time follows the slowest stage, not the sum of all stages.

VERIFICATION BEFORE RESULTS

Testbench evidence

Before benchmarking, the streaming kernel was checked against the reference pipeline at the same frame size.

307,200

pixels at 640 × 480

Exact compare

catches border, buffer and row-delay errors numerically

1

Stage references

each stage checked against a C++ model

2

Full-pipeline test

canny_top vs OpenCV-matching reference

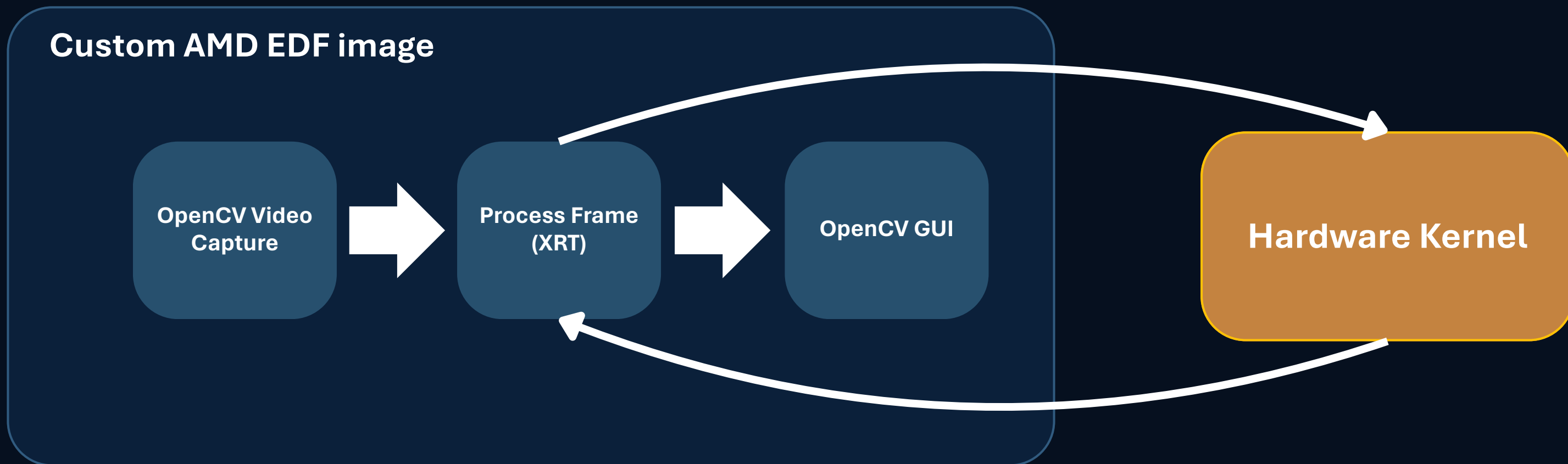
Validated before benchmarking: Native · Linux · Vitis CSim · synthesis

TRANSITION

Design question answered: the architecture is fixed and verified. Next, Luke measures board runtime, FPS, resources and speed-up against OpenCV.

Bridge to Luke: benchmark the stream kernel, compare to OpenCV, then discuss resource usage.

System Design: Software



Custom AMD EDF image

Petalinux was deprecated in 2025.2

EDF built using Yocto, incorporating extra packages (OpenCV, Matchbox)

Host Application

Built using Vitis, using custom SDK.

Captures webcam images using OpenCV, runs kernel using XRT, and displays output on OpenCV window

Results

3.680 ms

final runtime per frame

≈ 272 fps

Resource Usage

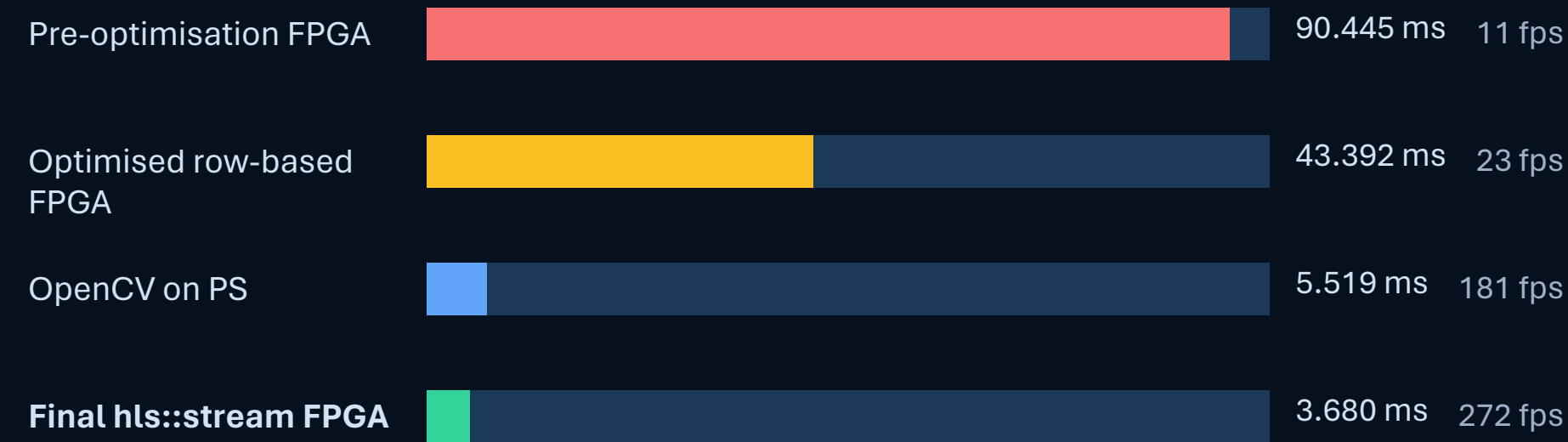
28%
BRAM

16%
LUT

4%
FF

~0%
DSP

Low resource usage leaves room for potential further optimizations.



1.50x Speedup over OpenCV

Achievable using hardware acceleration

Big breakthrough was switching to hls::stream between stages

Optimizing AXI-M bandwidth was also important to unlock the full potential of the hardware

Benchmarking Setup

Benchmark Inputs

640 x 480 static test image instead of using webcam.

Image is decoded using OpenCV from JPEG

Warmup Canny iteration + 1000 measured iterations

Benchmark Output

Output is verified on the last iteration for correctness

Xilinx Runtime profiling using xrt.ini was invaluable for optimization

OpenCV was run with max-frequency CPU governor across all 4 cores

Benchmark Input



Benchmark Output



Our Work

This project required us to work as a team to complete tasks and solve problems

Communication

Team was responsive and updates were posted regularly

Discord was the main tool of communication

Meetings were held online when needed

Workload

Workload was split evenly amongst teammates at the beginning of the project

Tasks were completed on time and each team member was able to work independently

Ai was used to streamline productivity

Github

Team worked together on the project using Github

New branches were created and changes reviewed to keep the workspace clean

Changes always passed the testbenches

GIVEN MORE TIME...

The team could have spent more time designing and chosen better designs earlier on

Optimisation could have been better discussed and executed

Thank you!

Any questions?